1 Advanced Python Concepts and Best Practices

Khwopa College of Engineering

What is Python?

Python is a **high-level, general-purpose programming language** known for:

- simple, readable syntax
- beginner-friendly design
- powerful standard library
- use across many industries

Key Features of Python

- **Easy to Read & Write** – clean, English-like syntax
- **High-Level** – abstracts low-level details
- **Interpreted** – runs line-by-line
- **Dynamically Typed** – no type declarations needed
- **Cross-Platform** – Windows, macOS, Linux

Simple Python Example

Hello World

```
print("Hello, world!")
```

Python Libraries

Python has an extensive ecosystem for:

- **Data Science**: NumPy, Pandas
- **Machine Learning**: TensorFlow, PyTorch
- **Web Development**: Django, Flask
- **Automation**: Selenium, PyAutoGUI

Where is Python Used?

Python is widely used in:

- Web development
- AI & machine learning
- Data analysis
- Automation and scripting
- Scientific computing
- Cybersecurity

Summary

- Python is versatile and easy to learn.
- Great for beginners and experts alike.
- Powers everything from small scripts to AI systems.

Python Data Types

Overview of Built-in Data Types in Python

Basic Data Types

Numeric Types

- `int` – whole numbers
- `float` – decimal numbers
- `complex` – numbers like `2 + 3j`

Boolean Type

- `bool` – `True` or `False`

Text Type

- `str` – string (text)

Sequence Types

Python supports several ordered collection types:

- `list` – mutable, ordered collection Example: `[1, 2, 3]`
- `tuple` – immutable, ordered collection Example: `(1, 2, 3)`
- `range` – sequence of numbers Example: `range(5)`

Set and Mapping Types

Set Types

- `set` – mutable, unordered collection of unique items Example: `{1, 2, 3}`
- `frozenset` – immutable version of a set

Mapping Type

- `dict` – key-value pairs Example: `{"name": "Alice", "age": 20}`

Binary and Special Types

Binary Types

- `bytes` – immutable byte sequences
- `bytearray` – mutable byte sequences
- `memoryview` – memory-access view of binary data

Special Type

- `NoneType` – represents "nothing"; value is `None`

Variables in Python

A **variable** is a name that stores a value.

- Think of it as a labeled container for data.
- Python automatically creates variables when you assign a value.

Creating Variables

- Assign a value to a name:

```
x = 10
name = "Alice"
is_valid = True
```

- No type declaration is needed — Python infers the type.

Variable Naming Rules

- Must start with a letter or underscore (_).
- Cannot start with a number.
- Can contain letters, numbers, and underscores.
- Case-sensitive: `age`, `Age`, `AGE` are different.
- Cannot use Python keywords (e.g., `for`, `if`).

Valid Examples: `total = 100`, $\texttt{user}_{n}ame = \text{"}Bob\text{"}$

Invalid Examples: `2x = 5`, `user-name = "Tom"`

Dynamic Typing & Multiple Assignments

Dynamic Typing:

```
num = 10
num = "now I am a string"
```

Multiple Assignments:

```
a, b, c = 1, 2, 3
x = y = z = 0
```

Check Type:

```
type(num)
```

Branching Statements in Python

Branching statements are used to make decisions in Python code.

- Execute certain blocks conditionally.
- The main statements: `if`, `if-else`, `if-elif-else`.

If Statement

Executes a block only if a condition is true:

```
x = 10
if x > 5:
    print("x is greater than 5")
```

If-Else Statement

Executes one block if the condition is true, another if false:

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is 5 or less")
```

If-Elif-Else and Nested If

If-Elif-Else:

```
x = 7
if x > 10:
    print("x > 10")
elif x > 5:
    print("x is 6 to 10")
else:
    print("x <= 5")
```

Nested If:

```
x = 8
if x > 5:
    if x % 2 == 0:
        print("x > 5 and even")
```

Conditional Expressions (Ternary Operator)

Shorthand one-line branching:

```
x = 10
result = "Even" if x % 2 == 0 else "Odd"
print(result)
```

- Combine conditions with `and`, `or`, `not`.
- Indentation is critical in Python.

Loops in Python

Loops are used to execute a block of code repeatedly.

- Two main types: `for` and `while`.
- Useful for iteration over sequences or conditional repetition.

For Loop

Iterates over a sequence (list, tuple, string, or range):

```
for i in range(5):
    print(i)
```

→ # 0, 1, 2, 3, 4

```
fruits = ["apple", "banana", "kiwi"]
for fruit in fruits:
    print(fruit)
```

While Loop

Repeats as long as a condition is True:

```
x = 0
while x < 5:
    print(x)
    x += 1
```

Loop Control Statements

- `break` – exits the loop immediately
- `continue` – skips the current iteration
- `else` – executes after loop ends normally

Example:

```
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("Done")
```

Nested Loops

Loops inside loops:

```
for i in range(3):
    for j in range(2):
        print(f"i={i}, j={j}")
```

- Useful for multidimensional data (like matrices).

Functions in Python

Functions are reusable blocks of code that perform a specific task.

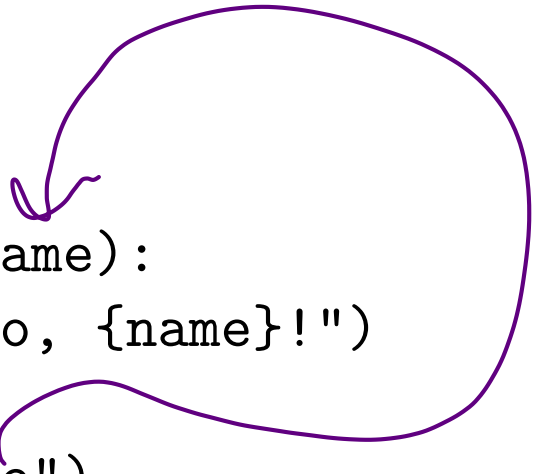
- Help organize code and avoid repetition.
- Defined using the `def` keyword.

Basic Function

```
def greet():
    print("Hello, World!")

greet()  # Calls the function
```

- Functions are executed only when called.

Functions with Parameters

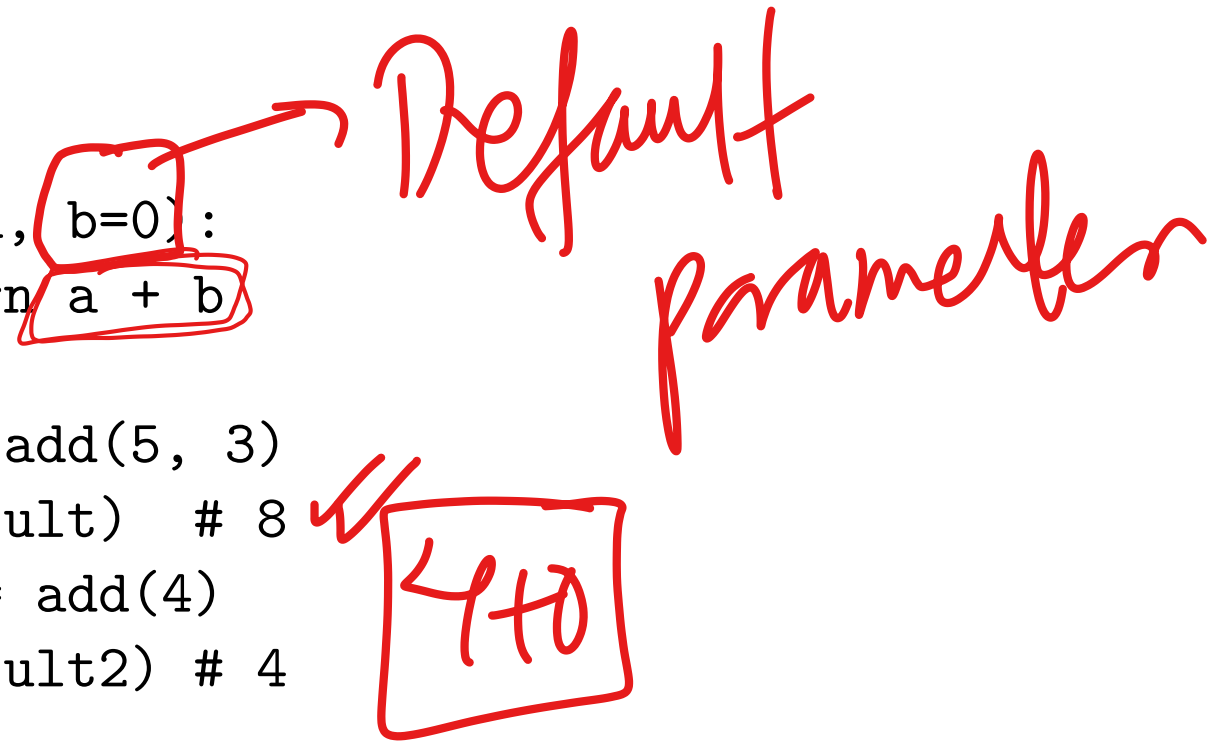


```
def greet_user(name):
    print(f"Hello, {name}!")

greet_user("Alice")
greet_user("Bob")
```

- Parameters allow passing values into functions.

Default Parameters and Return Values

```
def add(a, b=0):
    return a + b

result = add(5, 3)
print(result)  # 8
result2 = add(4)
print(result2) # 4
```



- Default parameters are used if no argument is provided.
- Functions can return values using `return`.

Variable Arguments (*args, **kwargs)

```
def show_info(*args, **kwargs):
    print("Args:", args)
    print("Kwargs:", kwargs)

show_info(1, 2, 3, name="Alice", age=25)
```

- `*args` – collects extra positional arguments.
- `**kwargs` – collects extra keyword arguments.

Python Coding Conventions (PEP 8)

Coding conventions help make Python code readable, consistent, and maintainable.

- PEP 8 is the official style guide for Python.
- Covers naming, indentation, line length, imports, and more.

Naming Conventions

- Variables / Functions: `snake_case`
  Example: `total_count`,
- `Classes:` `PascalCase`
  `Example:` `DataProcessor`
- `Constants:` `UPPER_CASE`
  `Example:` `MAX_SIZE`

- Be descriptive and meaningful.
- Case-sensitive: age $\neq$ Age

Indentation, Line Length, Blank Lines

Indentation: 4 spaces per level

```
if x > 0:
    print("Positive")
```

Line Length: Limit to 79 characters

```
total = (first_variable + second_variable +
         third_variable)
```

Blank Lines:

- 2 lines before top-level functions/classes
- 1 line between class methods

Imports

- Group imports in order:
  1. Standard library
  2. Third-party packages
  3. Local modules
- Separate groups with a blank line

```
import os
import sys

import requests

from .utils import helper
```

Other Best Practices

- Use spaces around operators: `x = 1 + 2`
- Avoid trailing whitespaces
- Use docstrings for functions, classes, and modules

Example:

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

Blank Lines in Python

- Use **2 blank lines** before top-level functions or classes.
- Use **1 blank line** between methods inside a class.

Example:

```
class MyClass:
    def __init__(self, value):
        self.value = value

    def display(self):
        print(self.value)


def my_function():
    print("Hello, World!")
```

Avoid Trailing Whitespace

- Trailing whitespace refers to extra spaces at the end of a line.
- PEP 8: Do not leave trailing whitespace.

Bad Example:

```
def greet():
    print("Hello, World!")
```

Good Example:

```
def greet():
    print("Hello, World!")
```

Use Docstrings for Documentation

- Docstrings describe functions, classes, or modules.
- Written in triple quotes: """ ... """

Function Example:

```
def add(a, b):
    """
    Add two numbers and return the result.
    """
    return a + b
```

Class Example:

```
class Person:
    """
    A class to represent a person.
    """
    def __init__(self, name, age):
        """Initialize a new Person instance."""
        self.name = name
```

Module Docstrings

```
"""
This module provides utility functions for mathematical operat

Functions:
- add(a, b): Return the sum of two numbers.
- subtract(a, b): Return the difference between two numbers.
"""
```

Key advanced features in Python include:

- **Collections:** Specialized container datatypes like `namedtuple`, `deque`, `Counter`, `defaultdict` (from the `collections` module)
- **Iterators:** Objects that can be iterated over using `iter()` and `next()`
- **Generators:** Functions using `yield` to produce items lazily
- **Decorators:** Functions that modify the behavior of other functions or classes

Collections in Python

Collections are specialized container datatypes that provide additional functionality beyond basic lists, tuples, and dictionaries.

- Found in the `collections` module
- Useful for counting, ordering, and managing data efficiently

namedtuple

`namedtuple` is like a tuple but with named fields:

```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20)
print(p.x, p.y)  # 10 20
```

deque (Double-Ended Queue)

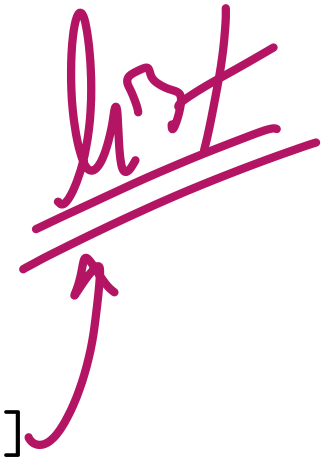
deque allows fast appends and pops from both ends:

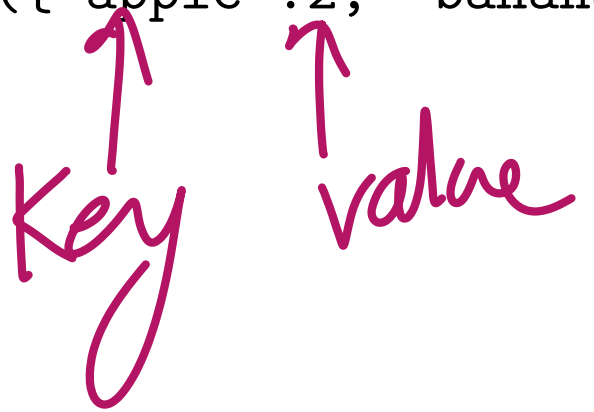
```
from collections import deque

dq = deque([1,2,3])
dq.append(4)       # Add to right
dq.appendleft(0)   # Add to left
dq.pop()           # Remove from right
dq.popleft()       # Remove from left
```

Counter

Counter counts occurrences of items:

```
from collections import Counter

words = ['apple','banana','apple','orange']
count = Counter(words)
print(count)  # Counter({'apple':2, 'banana':1, 'orange':1})
```





defaultdict

defaultdict provides default values for missing keys:

```
from collections import defaultdict

d = defaultdict(int)
d['a'] += 1
d['b'] += 2
print(d)  # defaultdict(<class 'int'>, {'a':1,'b':2})
```

Tuple vs Namedtuple

Comparison between standard tuples and namedtuples in Python

Differences Between Tuple and Namedtuple

- **Tuple:**
  - Immutable
  - Access by index: `t[0]`
  - Less readable for structured data
- **Namedtuple:**
  - Immutable
  - Access by name: `p.x` or by index
  - More readable and self-documenting

Example Program

```
from collections import namedtuple

# tuple
t = (10, 20)
print(t[0], t[1])

# namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20)
print(p.x, p.y)
```

Output

```
Tuple values: 10 20
Namedtuple values: 10 20
```

- Tuple values are accessed by index.
- Namedtuple values are accessed by name for readability.

List vs Deque in Python

Comparison between Python lists and deques

Differences Between List and Deque

- **List:**
  - Dynamic array
  - Fast random access by index
  - Inserting/removing at start is slow
- **Deque:**
  - Double-ended queue
  - Fast inserts/removals at both ends
  - Slower random access

Example Program

```
from collections import deque

# list
lst = [1, 2, 3]
lst.append(4)
lst.insert(0, 0)

# deque
dq = deque([1, 2, 3])
dq.append(4)
dq.appendleft(0)
```

Notes

- Use `list` for general purposes and fast indexing.
- Use `deque` for queue operations or frequent insertions/removals at both ends.
- `deque` is part of the `collections` module.

Iterators in Python

Iterators are objects that allow traversal over a collection of elements one at a time.

- Used in loops and comprehensions
- Implement $_{i}ter_{()}and$

Creating an Iterator

```
nums = [1, 2, 3]
it = iter(nums)   # Create iterator
```

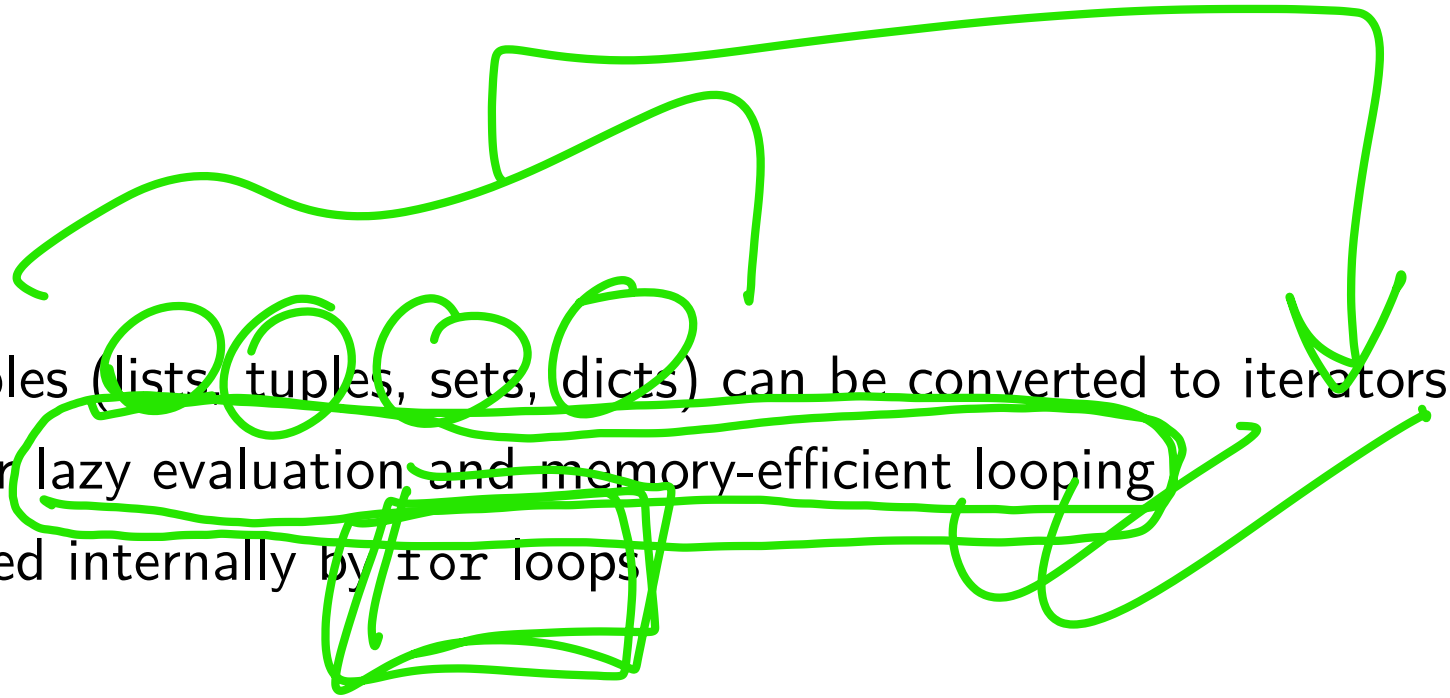
- Use `iter()` to get an iterator from any iterable

Using next()

```
print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
# print(next(it))  # Raises StopIteration
```

- Each call to `next()` retrieves the next element
- Stops iteration when all elements are exhausted

Notes on Iterators



- All iterables (lists, tuples, sets, dicts) can be converted to iterators
- Useful for lazy evaluation and memory-efficient looping
- Often used internally by `for` loops

Generators in Python

Generators are functions that produce values lazily, yielding one item at a time.

- Use `yield` instead of `return`
- More memory-efficient than returning a full list

Generator Function Example

```
def my_generator(n):
    for i in range(n):
        yield i

gen = my_generator(3)
```

- gen is a generator object

Using next()

```
print(next(gen))  # 0
print(next(gen))  # 1
print(next(gen))  # 2
# print(next(gen))  # Raises StopIteration
```

- Each call to `next()` yields the next value
- Stops iteration when all items are generated